

# Core Design Principles

Software Design (40007) – 2023/2024

Justus Bogner  
Ivano Malavolta

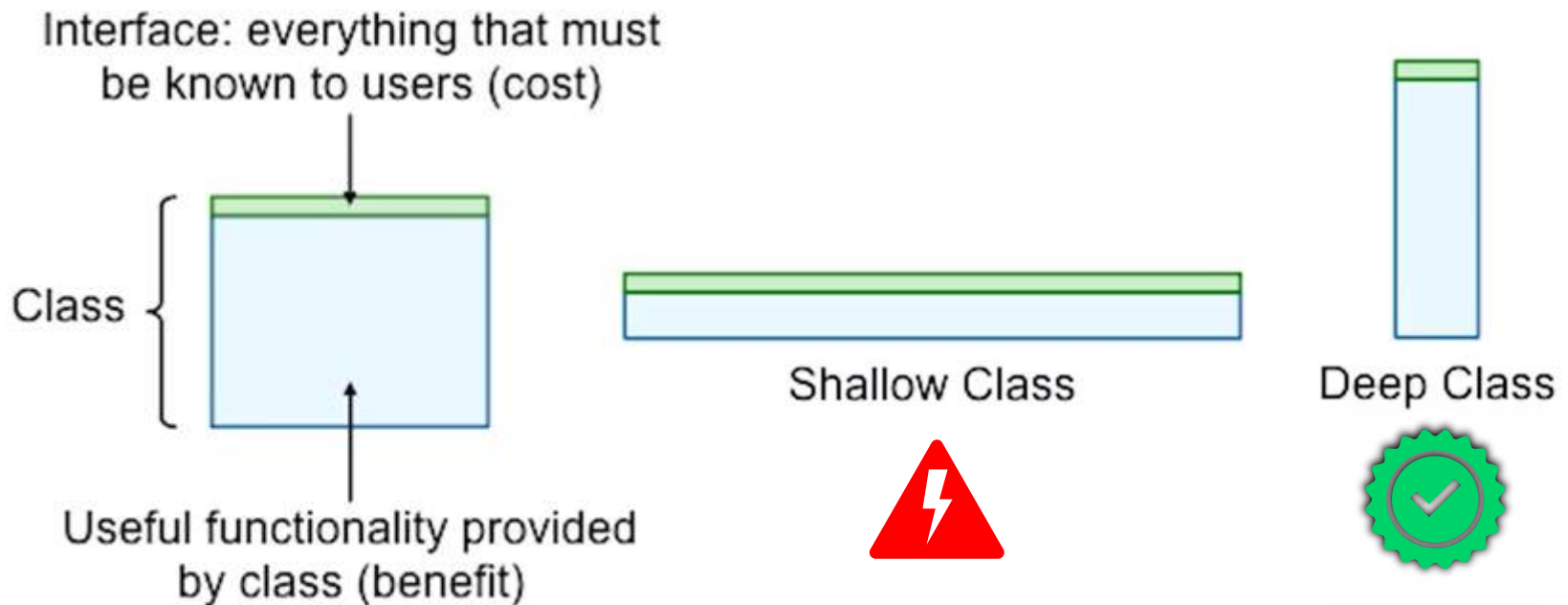
# Roadmap

---

- The single responsibility principle (SRP)
- Encapsulation & immutability
- Avoid complexity (or *design for simplicity*)

# How to structure your code (and class diagram)

- Strive towards having **deep classes**



# Information hiding and operation usability

---

- Minimize the **information needs** of each class
    - Prioritize information hiding
    - `private` is your default
    - `getter()` and `setter()` methods only when needed
  - Focus on the **usability** of the operations of each class
    - Exposed APIs should be easily and intuitively understandable
    - When adding an operation/parameter, think if it is really needed by the rest of the system
    - Push complexity as low as possible in the class diagram hierarchy
- A **deep class** hides less relevant information / complexity and provides valuable and easily usable operations.

# **The Single Responsibility Principle (SRP)**

# What is the single responsibility principle (SRP)?

---

Design classes in such a way that there is **only a single reason to change a class**.

- Leads to smaller and more cohesive classes
- Leads to less complex classes

→ Classes that are easier to understand and change

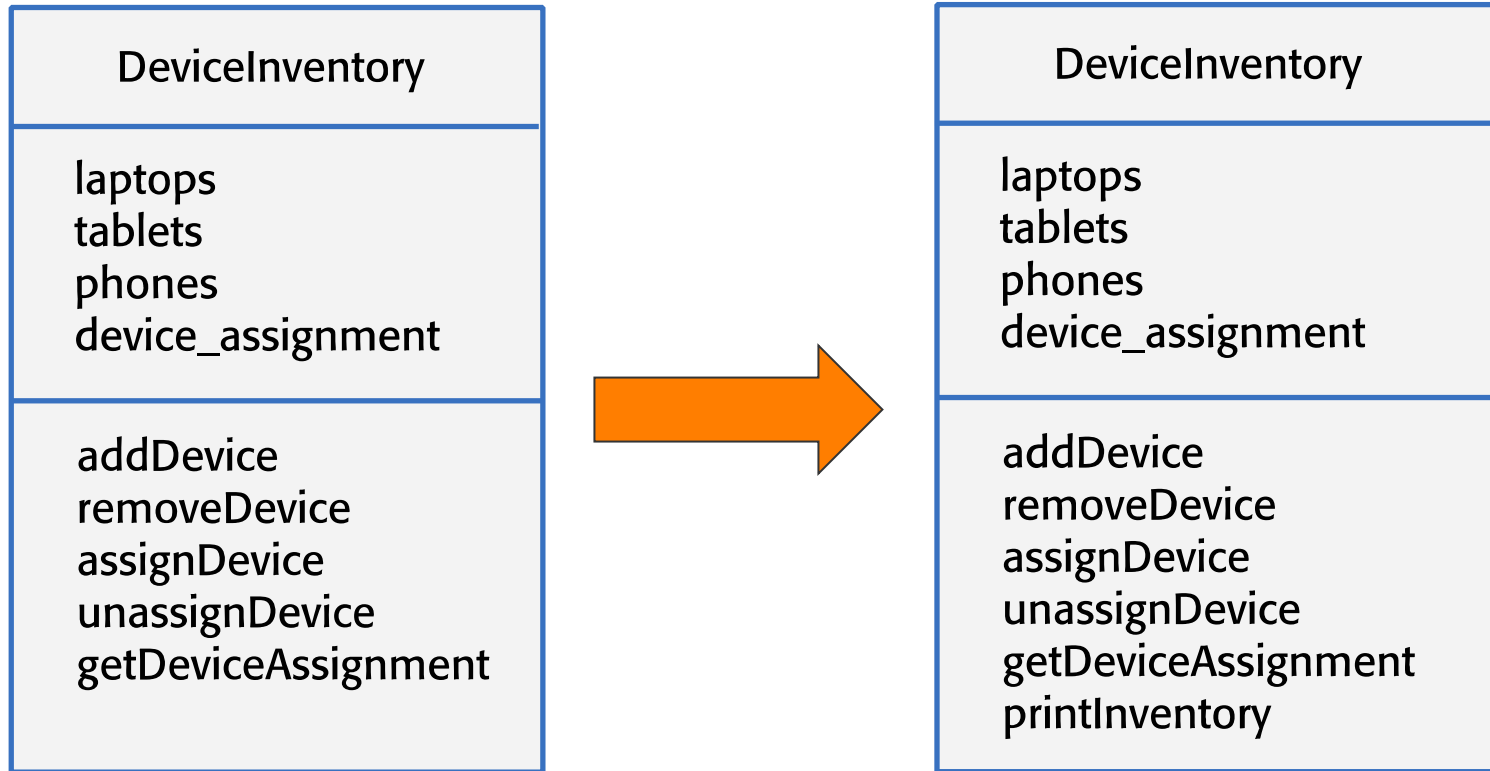
■ Therefore:

- Group entities\* that change for the same reasons
- Separate entities that change for different reasons

→ **Functional cohesion**

\* *Entity = class, method, attribute*

# Example: violating SRP



# Example: preserving SRP

---

| DeviceInventory                                                                    |
|------------------------------------------------------------------------------------|
| laptops<br>tablets<br>phones<br>device_assignment                                  |
| addDevice<br>removeDevice<br>assignDevice<br>unassignDevice<br>getDeviceAssignment |

| InventoryReport                     |
|-------------------------------------|
| report_data<br>report_format        |
| updateData<br>updateFormat<br>print |

How are we going to "link" those two classes?



# Encapsulation & Immutability

# What is encapsulation?

---

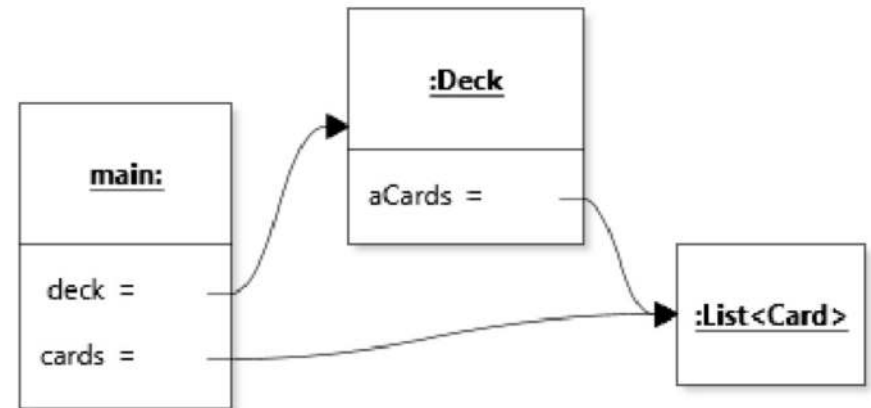
- The act of keeping both the data and the computation together to limit the number of contact points between different parts of your system
- Closely related to information hiding
- Advantages:
  - Understanding a piece of code in isolation is easier
  - Using a piece of code becomes less error-prone
  - Changing a piece of code less likely breaks something else

Encapsulation is often violated through references that escape.

# Escaping references

```
1 public class Card {
2     private Rank aRank;
3     private Suit aSuit;
4
5     public Card(Rank pRank, Suit pSuit) {
6         aRank = pRank;
7         aSuit = pSuit;
8     }
9
10    public Rank getRank() {
11        return aRank;
12    }
13
14    public Suit getSuit() {
15        return aSuit;
16    }
17 }
```

```
1 Deck deck = new Deck();
2 List<Card> cards = deck.getCards();
3 cards.add(new Card(Rank.ACE, Suit.HEARTS));
```



```
1 public class Deck {
2     private List<Card> aCards = new ArrayList<>();
3
4     public Deck() {
5         // add and shuffle cards
6     }
7
8     public Card draw() {
9         return this.aCards.remove(0);
10    }
11
12    public List<Card> getCards() {
13        return this.aCards;
14    }
15 }
16
```

# How references escape

---

There are 3 ways in which references can escape:

1. Returning a reference to an external object
  - See previous slide
2. Storing an external reference internally
3. Leaking a reference through a shared structure
  - Similar to the previous ones but indirect, e.g., lists of lists

Example of 2:

```
1 public class Deck {
2     private List<Card> aCards = new ArrayList<>();
3
4     public Deck() {
5         // add and shuffle cards
6     }
7
8     public void setCards(List<Card> cards) {
9         this.aCards = cards;
10    }
11 }
12
13 // ...
14
15 List<Card> cards = new ArrayList<>();
16 Deck deck = new Deck();
17 deck.setCards(cards);
18 cards.add(new Card(Rank.ACE, Suit.HEARTS));
19
```

# Immutability

---

Objects are **immutable** if their class provides no way to change the internal state of the object after instantiation.

Immutable class = a class that yields immutable objects

## Advantages:

- You can share information without breaking encapsulation
- You avoid temporal method dependencies (invocation order)
- It leads to thread safety
- It allows the caching of objects

## Disadvantage:

- You tend to create more objects
- Decreased performance efficiency (more garbage collection)

In Java: primitive types and enumerations are immutable by default

- Some other cases in Java libraries (see documentation)

# How to make objects immutable

---

- Ensure that all the fields of your class are:
  - Either `private` and not changed by any instance method
  - Or immutable by default
    - Primitive types or enumerations
    - `final`
- Expose internal data consciously
  - Extended interface
  - Return copies
  - Via dedicated design patterns (covered later in the course)

# Extended interfaces

---

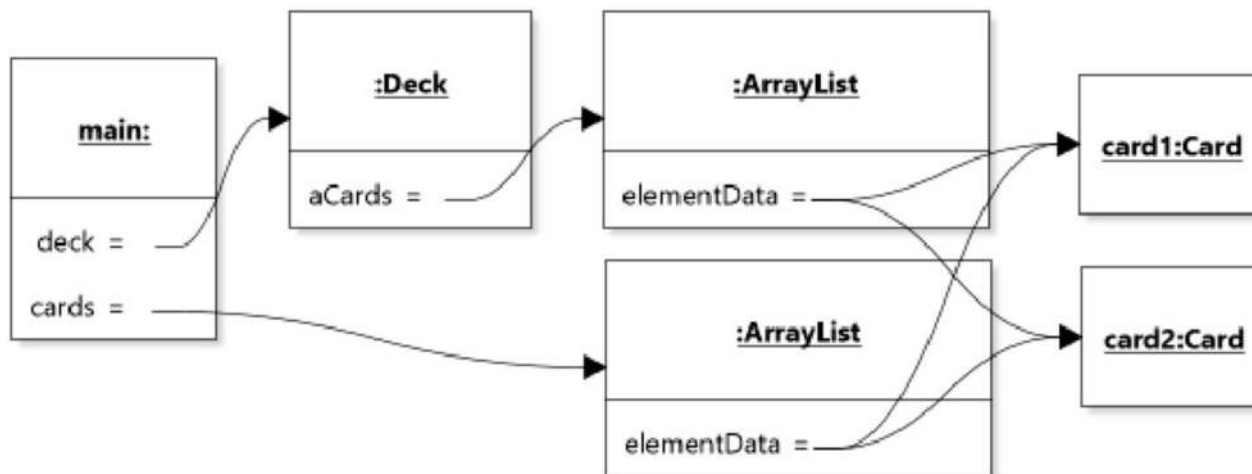
You extend the interface of the class, i.e., its set of `public` methods, with methods returning only references to immutable objects.

```
1 public class Deck {
2     private List<Card> aCards = new ArrayList<>();
3
4     public int size() {
5         return this.aCards.size();
6     }
7
8     public Card getCard(int index) {
9         //assuming Card is immutable
10        return this.aCards.get(index);
11    }
12
13 }
```

# Returning object copies

You internally **clone** the stored object and return the newly created copy instead of the original.

```
1 public class Deck {  
2     private List<Card> aCards = new ArrayList<>();  
3  
4     public Card getCards() {  
5         //assuming Card is immutable  
6         return new ArrayList<>(this.aCards);  
7     }  
8 }
```





# Knowing how to copy

---

In the previous example, we are trusting the implementation of the constructor of `ArrayList`.

- Always check the documentation of the methods you are calling!
- This could have led to the same result:

```
1 public List<Card> getCards() {  
2     return Collections.unmodifiableList(this.aCards);  
3 }
```

**PROBLEM:** how deep should we copy objects?

**ANSWER:** until we reach immutable referenced objects

# Copy constructors

A popular technique for copying objects is to use a **copy constructor**.

```
1 public class Card {
2     private Rank rank;
3     private Suit suit;
4
5     public Card(Card pCard) {
6         this.rank = pCard.rank;
7         this.suit = pCard.suit;
8     }
9 }
10
11 public class Deck {
12
13     // ...
14
15     public List<Card> getCards() {
16         ArrayList<Card> result = new ArrayList<>();
17         for(Card card : this.aCards) {
18             result.add(new Card (card )
19         }
20         return result;
21     }
22 }
```

**Avoid Complexity /  
Design for Simplicity**

# What is complexity?

Practical definition from (Ousterhout, 2018):

«anything related to the structure of a software system that makes it **hard to understand and modify** the system.»

- **Inherent complexity**

Unavoidable domain complexity

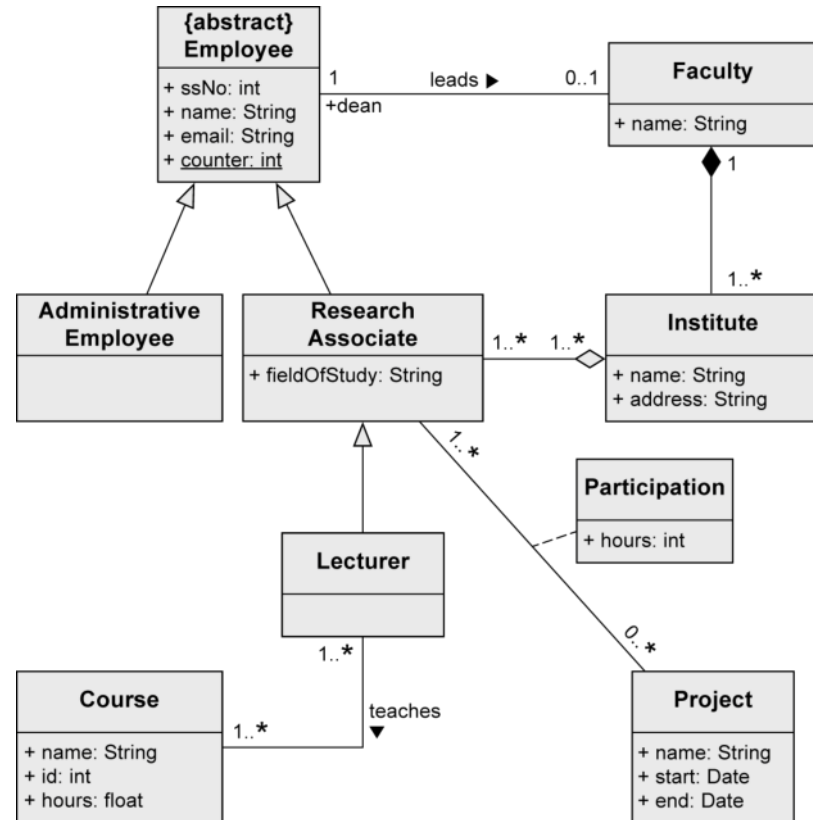
vs.

- **Accidental complexity**

Avoidable technical complexity introduced through suboptimal design

## Two general strategies:

- Encapsulate inherent complexity
- Reduce accidental complexity through good, simple design



# Types of complexity

---

- **Structural complexity [= coupling]**

The number and strength of relationship between the structures in your program (packages, classes, methods)

- **Reading complexity**

How hard it is to read and understand the program

- **Data complexity**

The data representations and relationships between the data elements in your program

- **Decision complexity**

The complexity of the decision flows in your program

Which of these can you  
influence **the least**?

# Guidelines for reducing complexity (1)

---

- **Structural complexity**

- Methods should do one thing and one thing only
- Every class should have a single responsibility
- Methods should not have side-effects
- Minimize the depth of inheritance hierarchies
- Avoid multiple inheritance
- Avoid threads (parallelism) unless absolutely necessary

} SRP

# Guidelines for reducing complexity (2)

---

- **Data complexity**

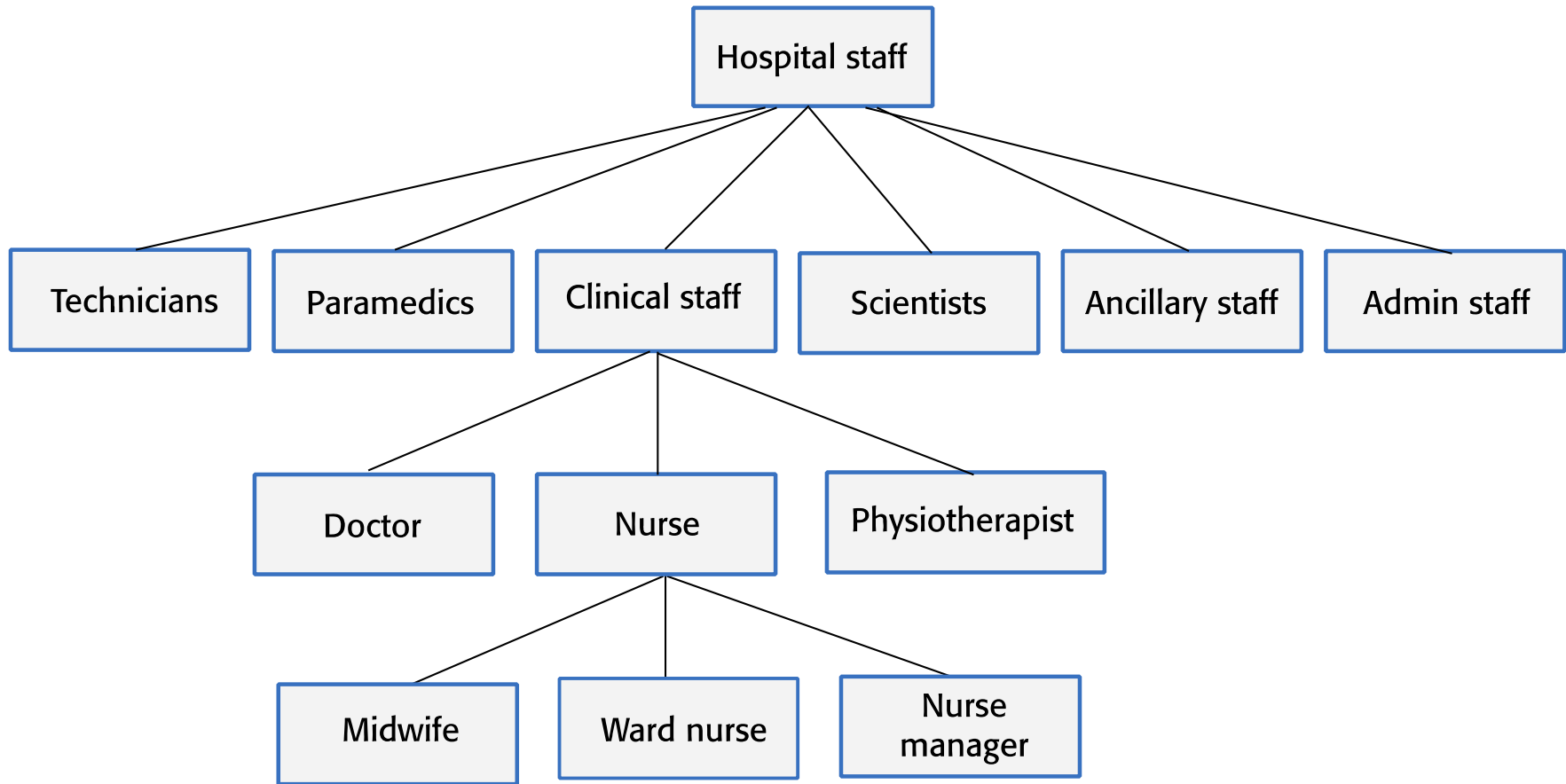
- Define understandable interfaces for important abstractions
- Define abstract data types if it substantially reduces duplication
- Avoid using floating-point numbers if possible [1]

- **Decision complexity**

- Avoid deeply nested conditional statements
- Avoid complex conditional expressions, e.g., extract parts to functions with clearly understandable names

[1] <https://floating-point-gui.de>

# Minimize the depth of inheritance hierarchies



Why is deep hierarchy a problem?

What happens if we “flatten” this hierarchy?